

ALGORITHME correcteur_orthographique(texte)

ENTREE: texte (chaîne de caractères)

SORTIE: liste de corrections

// Étape 1: Tokenisation

mots = tokeniser(texte)

corrections = liste_vide()

POUR CHAQUE mot DANS mots FAIRE

// Étape 2: Vérification dans le dictionnaire

SI mot $\notin$ dictionnaire ET longueur(mot) > 2 ALORS

// Étape 3: Calcul de similarité

suggestions = calculer_similarites(mot, dictionnaire)

// Étape 4: Vérification phonotactique

erreurs_phonotactiques = verifier_phonotactique(mot)

// Étape 5: Score final

SI suggestions $\neq \emptyset$ OU erreurs_phonotactiques $\neq \emptyset$ ALORS

ajouter_aux_corrections(corrections, mot, suggestions, erreurs_phonotactiques)

FIN SI

FIN SI

FIN POUR

RETOURNER corrections

FIN ALGORITHME

ALGORITHME calculer_similarites(mot, dictionnaire)

ENTREE: mot, dictionnaire

SORTIE: dictionnaire de suggestions avec scores

suggestions = dictionnaire_vide()

POUR CHAQUE mot_dico DANS dictionnaire FAIRE

// Distance de Levenshtein optimisée

score = rapidfuzz_ratio(mot, mot_dico)

SI score > SEUIL_SIMILARITE (60) ALORS

suggestions[mot_dico] = score

FIN SI

FIN POUR

// Tri par score décroissant

suggestions_triees = trier_par_valeur(suggestions, ordre='desc')

RETOURNER suggestions_triees

FIN ALGORITHME

ALGORITHME verifier_phonotactique(mot)

ENTREE: mot

SORTIE: liste d'erreurs phonotactiques

erreurs = liste_vide()

regles = [

r'\bnb', r'\bnk', r'\bmk', // Interdits début

r'dt\$', r'bp\$', r'sz\$', // Interdits fin

r'([aeiou])\1{2,}' // Voyelles triples

]

POUR CHAQUE regle DANS regles FAIRE

SI regex_match(regle, mot) ALORS

message = obtenir_message_erreur(regle)

ajouter(erreurs, message)

FIN SI

FIN POUR

RETOURNER erreurs

FIN ALGORITHME